

CODE SPORT



In this month's column, we continue our discussion on deep learning.

A few months back, we had started a discussion on deep learning and its application in natural language processing. However, over the last couple of months, we digressed a bit due to various other requests from our readers. From this month onwards, let's get back to exploring deep learning. In prior columns, we had discussed how supervised learning techniques have been used to train neural networks. These learning techniques require labelled data. Given that today's state-of-art neural networks are extremely deep, with many layers, we require large amounts of data so that we can train these deep networks without over-fitting. Getting labelled annotated data is not easy. For instance, in image recognition tasks, we need to have specific pieces of images bound together in order to recognise human faces or animals. Labelling millions of images requires considerable human effort. On the other hand, if we decide to use smaller amounts of labelled data, we end up with over-fitting and poor performance on test data. This leads to a situation wherein many of the problems, for which deep learning is an ideal solution, do not get tackled simply due to the lack of large volumes of labelled data. So the question is: is it possible to build deep learning systems based on unsupervised learning techniques?

In last month's column, we had discussed word embeddings, which along with paragraph and document embeddings are now widely used as inputs in various natural language processing tasks. In fact, they are used as input representation for many of the deep learning based text processing systems. If you have gone through the word2vec paper at <https://papers.nips.cc/paper/5021-distributed-representations-of-words-and-phrases-and-their-compositionality.pdf>, you would have seen that there is no labelled training data for the neural network described in this paper. The word2vec neural network is not a deep neural network. There are only three layers — an input layer, a hidden layer and

an output layer. Just as in the case of supervised learning neural networks, back propagation is used to train the weights of the neural network. Here is a quick question to our readers — why do we initialise the weights of the nodes to be random weights instead of initialising them to zero?

In a supervised learning setting, we have labelled data, which provides what is the correct/expected output/label for a given input. The neural network produces an actual output for a given input. The difference between expected output and actual output is the error term at the output layer. As we had seen earlier during the discussion on back-propagation, this error term is back-propagated to earlier layer nodes whose weights are determined based on these error terms. The key idea behind back propagation is the following — weights of each node are adjusted in proportion to how much it contributes to the error terms of the next layer's nodes, for which the first node's output acts as input. For back propagation to work, the desired output for a given input for each output layer's node needs to be known. This can then be back-propagated to the hidden layer's neurons. Here is a question to our readers — in the word2vec paper, where the neural network is being trained, what is the expected output for a given input?

Well, the answer is simple. The expected output that is used for training is the same as the input itself. If you consider the simplest variant of the word2vec continuous bag-of-words model, with only one word per context, the task is to predict the target word, given one word in context. If we make the assumption that the context word and target word are adjacent, this reduces to the bigram model, and we need to predict the succeeding word given a word in a continuously moving window of the corpus. This task requires no explicitly labelled data except the input corpus itself. Thus, word2vec is a great example for a neural network method which can scale to large data, but does not require explicit labelled data.